Presentation Slides — DevSecOps Track 2 Assessment

SLIDE 1 — Title

DevSecOps Track 2
Multi-Cloud Platform Security & Compliance

Security Review — TerraGoat Reference Repository
AWS | Azure | GCP — Terraform IaC

Date: 2026-07-09
Repository: github.com/rajutest-devops/devsecops-assessment-track2

SLIDE 2 — Agenda (25 minutes)

Time	Topic
0-3 min	Executive risk summary
3-8 min	Top findings + why prioritised
8-13 min	Identity, Network, Data Protection (how they connect)
13-17 min	Architecture: HLD + LLD
17-20 min	COTS compensating controls recommendation
20-23 min	Pipeline + detection design
23-25 min	First 90 days — what I'd actually do

SLIDE 3 — Executive Risk Summary

Overall Posture: HIGH

Severity	Count	Actively Exploitable	Time-to-Exploit
Critical	4	Yes	< 1 hour
High	16	Likely	1-24 hours

3 clouds assessed: AWS (12 findings) · Azure (6 findings) · GCP (2 findings)
4 tools used: Checkov · TFSec · Trivy · GitLeaks
445+ raw checks → 20 curated, exploitable findings

> *"The most urgent risk is hardcoded AWS credentials in source code. Anyone with read access to the repository has access to the AWS account. This does not require any technical skill to exploit."*

SLIDE 4 — Top 4 Critical Findings

ID	Finding	File	Why #1 Priority
FIND-001	AWS access key hardcoded	`ec2.tf:5`	Anyone with repo access = AWS access. Permanent.
FIND-002	AWS secret key in Lambda	`lambda.tf:15`	Same account, different service. Compounds FIND-001.
FIND-004	S3 bucket publicly readable	`s3.tf:22`	PII data accessible to internet. No auth required.
FIND-003	JWT token sent to webhook.site	`bla.yml:7`	CI/CD credentials exfiltrated to public endpoint.

Why these four above everything else:

- FIND-001/002: Zero prerequisites. Google the AWS key format → found. Run `aws sts get-caller-identity` → confirmed.
- FIND-004: S3 bucket enumeration tool finds it in minutes. Data breach, no alert fires.
- FIND-003: Token sent to webhook.site = public log. Attacker replays it to GitLab API = infrastructure modification rights.

SLIDE 5 — Prioritisation Methodology

How 20 findings were selected from 445+ raw check failures:

```
445 raw Checkov failures
    ↓
Remove: Informational/Low (not exploitable) = −180
    ↓
Remove: Duplicates (same issue, different file) = −95
    ↓
Remove: False positives (test fixtures, intentional) = −80
    ↓
Remove: Compliance-only (no active exploit path) = −70
    ↓
20 findings with: confirmed exploit path + real-world impact
```

Severity = Exploitability × Business Impact

Exploitability	Impact	Severity
High (no skills needed)	Critical (account takeover)	**Critical**
Medium (some skill)	High (data breach)	**High**
Low (expert needed)	Medium (partial access)	**Medium**

SLIDE 6 — Identity, Network & Data: One Story

The three designs work together — not independently:

```
Attacker path without fixes:
Find key in git (5 min) → Access AWS → List public S3 → Download PII → Exit undetected

After remediation:
Find key in git → Key is a role (no secret) ✗ BLOCKED
  OR
Find open port 22 → SSH blocked (SSM only) ✗ BLOCKED
  OR
Find public S3 → Bucket is private ✗ BLOCKED
  AND
All attempts logged to CloudWatch → Alert fires within 5 min
```

Identity (Module 03): Removes permanent credentials → roles + OIDC only
Network (Module 04): Removes attack surface → private subnets + no SSH
Data (Module 05): Removes data accessibility → KMS encryption + private buckets
Detection (Module 06): Detects if all three above fail → SIEM + alerts

---

SLIDE 7 — High-Level Architecture

(Reference: 10-architecture/hld-diagram.md)

Key layers (top to bottom):

```
[External Users]
    ↓ HTTPS
[Edge: WAF + CloudFront]          ← Blocks DDoS, OWASP attacks
    ↓ Filtered
[App Tier: Private Subnets]       ← No public IPs, SSM access only
    ↓ Port 5432 (SSL required)
[Data Tier: Encrypted Storage]    ← KMS at rest, TLS in transit
    ↓
[Security: KMS + Secrets Manager] ← Keys never in code

[CI/CD: GitHub Actions]
    ↓ OIDC token (1-hour, no keys)
[AWS / Azure / GCP]               ← Deploy without stored credentials

[All clouds] → [Centralised SIEM] ← Unified detection across 3 clouds
```

3 trust boundaries:

- Public internet → Edge (WAF filters)
- Edge → App (private subnet, no direct path)
- App → Data (security group: app-sg only, specific port)

SLIDE 8 — Low-Level Architecture (AWS Slice)

(Reference: 10-architecture/lld-diagram.md)

The path from user to database:

Layer	Component	Security Config
Edge	CloudFront + WAF	TLS 1.2+, OWASP rules, rate limiting
Load Balancer	ALB	HTTPS only, HTTP → 301 redirect
App	EC2 (IMDSv2)	IAM role, no keys, SSM-only admin
Database	RDS Aurora	`force_ssl=1`, private subnet, no public IP
Keys	AWS KMS	Annual rotation, audit in CloudTrail
Admin	SSM Session Manager	No port 22, logged to CloudTrail

CI/CD path:

- Developer pushes → Checkov + TFSec scan → GitLeaks scan → manual approval → terraform apply (OIDC, no stored keys)

SLIDE 9 — COTS Compensating Controls Recommendation

The constraint: Financial reconciliation tool cannot support encryption-at-rest. Vendor limitation. Cannot re-platform before compliance deadline.

Three compensating controls deployed:

Control	What It Does	Replaces Encryption By...
CC-001: Network isolation	Private subnet, no egress, app-only access	Raising attack bar: must breach subnet first
CC-002: Full audit logging	Every query logged, immutable archive, bulk-export alerts	Detects breach within 5 min (vs weeks)
CC-003: TLS + OS access control	SSL-only connections, no SSH to host	Protects data in transit, blocks disk mounting

Residual risk after controls: Medium (down from Critical)

Panel decision required:

> *"This is a risk decision, not a technical one. The recommendation is: accept interim risk with these three controls, signed by ITSO + Data Owner, with a hard Q2 re-platform commitment. If the vendor cannot deliver an encrypted driver by Q2, the decision escalates to the programme board."*

SLIDE 10 — Pipeline & Shift-Left Design

What the secure pipeline catches before merge:

```
Push / Pull Request
    ↓
[terraform fmt + validate]  ← Syntax errors caught in < 30 sec
    ↓
```

```
[Checkov + TFSec]          ← IaC misconfigs: CIS/PCI mapped, CRITICAL blocks merge
    ↓
[GitLeaks]                 ← Secrets in code: BLOCKS merge (allow_failure: false)
    ↓
[PR Comment: status table] ← Developer sees results without leaving GitHub
    ↓
[Manual approval gate]     ← Security sign-off before production
```

What the original pipeline was doing:

- SC-001: Sending CI job token to public webhook (credential exfiltration)
- SC-002: Self-hosted runner on public repo (code execution risk)
- SC-003: Unpinned `@main` actions (supply chain attack vector)
- SC-004: Python 3.7 EOL (no security patches since June 2023)
- SC-005: Checkov running but never blocking (findings ignored)

SLIDE 11 — Detection & Monitoring

Current state: No centralised detection. Some accounts have CloudTrail, others don't.

Target state:

Cloud	Logs Collected	Destination	Alert On
AWS	CloudTrail, VPC Flow, EKS audit	CloudWatch → S3 (Object Lock)	Unauthorised API calls, bulk S3 download
Azure	Azure AD, AKS audit, Activity	Log Analytics	Failed MFA, admin outside hours
GCP	Cloud Audit, GKE control plane	Cloud Logging → BigQuery	kubectl exec, service account key creation

Key design decision: Logs go to immutable storage (S3 Object Lock, 7-year WORM)
→ Even a compromised admin account cannot delete them

Incident scenario: Wildcard CI service account compromised, used to access database

1. **Detect (0-5 min):** CloudTrail fires alert on `sts:AssumeRole` from unknown IP
2. **Contain (5-60 min):** Revoke CI role, block IP, enable verbose logging
3. **Eradicate (1-4 hrs):** Rotate all credentials, patch vulnerability, audit access
4. **Communicate:** Page ITSO within 15 minutes. Post-incident report within 48 hours.

SLIDE 12 — Compliance Coverage

5 frameworks mapped against 20 findings:

Framework	Controls Assessed	Failures	Status
CIS AWS Foundations	12	8	Non-compliant
CIS Azure Security	8	5	Non-compliant

CIS GCP Benchmark	9	4	Non-compliant
PCI-DSS v4.0	5 requirements	3	Non-compliant
NIST CSF	All 5 functions	3 weaknesses	Partial

Remediation impact:

- After P0 fixes (FIND-001/002/004/005): PCI-DSS Req 3 + 7 satisfied
- After P1 fixes: CIS AWS Foundations 90% compliant
- After P2 fixes: Azure + GCP CIS compliant
- Full compliance: ~8 weeks of remediation work

SLIDE 13 — First 90 Days on the Programme

What I'd actually do, in order:

Week 1-2 (Immediate risk reduction):

- Deploy P0 fixes: Remove hardcoded credentials + IAM roles + S3 lockdown
- Deploy COTS compensating controls (CC-001 + CC-002 + CC-003)
- Enable CloudTrail + basic alerting on all AWS accounts
- Get ITSO signature on COTS risk acceptance form

Week 3-4 (Shift-left pipeline):

- Deploy Checkov + TFSec + GitLeaks to GitLab CI (across all 100 workloads)
- Triage existing findings: assign owners, set P1-P3 deadlines
- Stand up centralised log aggregation (start with AWS, add Azure/GCP)

Month 2 (Governance and breadth):

- Roll out MFA enforcement via Conditional Access
- Restrict SSH/RDP to bastion/SSM
- Begin IAM least-privilege audit (wildcard permissions)

Month 3 (Detection and compliance):

- CSPM continuous scanning across all accounts
- Compliance report: what's fixed, what's residual risk, what has exception
- Architecture review with engineering leads: target state roadmap for Azure/GCP onboarding

Not done in 90 days: Full ZTA, service mesh, cross-cloud failover — these are 6-12 month programmes.

SLIDE 14 — Summary: Risk Decisions for the Panel

3 decisions the panel needs to make:

Decision	Recommendation	Risk If Deferred
1. COTS exception	Accept with 3 compensating controls + Q2 re-platform commitment	Current state: Critical unmitigated

2. Compliance deadline	P0 fixes (24 hrs) + P1 (1 week) achievable before go-live	Missing deadline = failed sign-off
3. Remediation funding	~80 hours engineering time over 4 weeks	Findings remain exploitable

What I need from the panel:

- ITSO signature on COTS risk acceptance
- Engineering time allocation: 4 weeks, 2 engineers
- Mandate to enforce pipeline scanning across all 100 workloads (not opt-in)

SLIDE 15 — Questions

Common questions I'm prepared for:

- "Why these 20 findings and not the other 425?" → Severity methodology: exploitability × impact, documented in 01-findings/severity-methodology.md
- "How long will remediation actually take?" → P0: 1 day. P1: 1 week. Full: 4 weeks. ~80 hours engineering.
- "What if the COTS vendor never fixes the driver?" → Time-bound exception with quarterly review. After Q2, decision goes to programme board.
- "How does the pipeline scale across 100 workloads?" → GitLab group-level CI template. One config, all projects inherit it. Risk-based: critical workloads get additional manual gate.
- "What's the biggest risk you're not fully mitigating?" → COTS encryption gap. All others have direct technical fixes. That one requires business decision.

Repository: github.com/rajutest-devops/devsecops-assessment-track2
Prepared for: DevSecOps Track 2 Technical Assessment